AI-Assisted Software Development

Harnesses, Frontier Models, Common Practices, Issues, and What's Possible

Bill Katz
Scientific Computing Software
HHMI Janelia Research Campus

Companion Resource Page

This talk has a repo: `github.com/JaneliaSciComp/ai-assisted-dev`

In addition to this presentation which uses Slidev, there's a `resources` directory with an Awesome-style markdown page of links to AI-assisted software development resources.

It's in early development – mostly seeded with sources from this talk. The goal is a curated, searchable collection of links for AI-assisted development at Janelia.

If time permits – some wild ideas for where this could go:

- Rate and comment on resources (short one-liner Janelian reviews)
- Tag-based search across tools, approaches, and examples
- "Follow" individual Janelians to see what they find useful
- AI-powered search that combines the above to suggest resources

For now: explore the links, and contribute anything you've found helpful.

Talk Roadmap

1. **The Ecosystem** – Frontier models, harnesses, and what's available at Janelia
2. **Approaches** – Common patterns for AI-assisted development
3. **Issues** – Context rot, model degradation, and the feedback loop
4. **Extraordinary Examples** – What people are actually building

Part I: The Ecosystem

Frontier Models & Harnesses

Three Frontier Systems

	Claude (Anthropic)	**GPT / Codex** (OpenAI)	**Gemini** (Google)
Model	Opus 4.6, Sonnet 4.6	GPT-5.4	3.1 Pro, Flash
Context	200K-1M tokens	200K-1M	1M tokens
Harness	**Claude Code & App**, **Cowork**	**Codex**, **ChatGPT**	**Gemini CLI**, **Antigravity**

All support tool use, MCP, and agentic workflows.

Introduced: Claude Code (Feb 2025), OpenAI Codex (May 2025), Gemini CLI (June 2025)

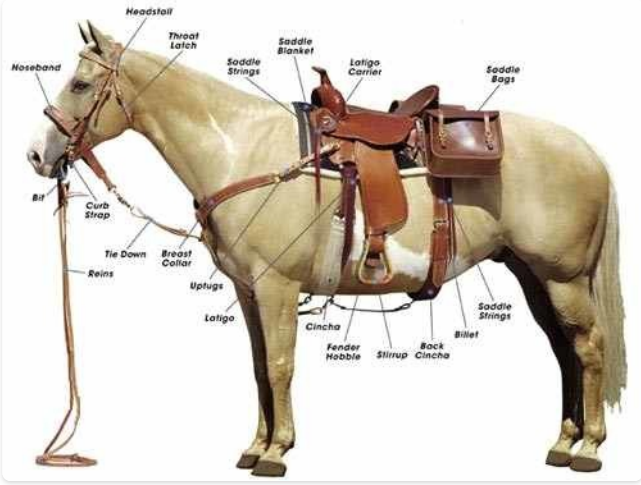
Early 2026: Agentic systems (run in parallel) with advances in both models and harnesses, have **changed** the software development landscape.

What is a Harness?

The **model** is the horse – raw power and capability.

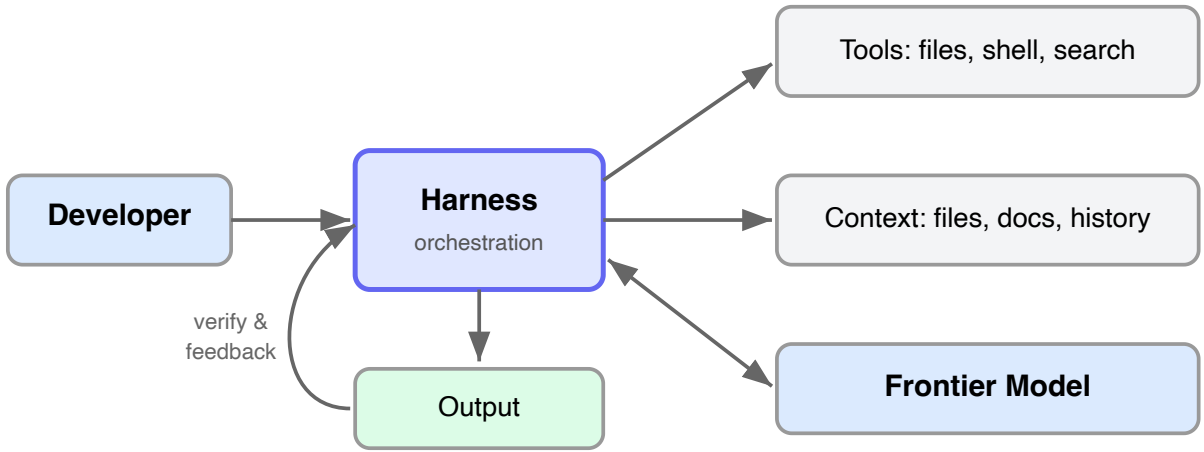
The **harness** is the equipment that lets you direct the horse, keep the rider comfortable, and make the trip productive.

Saddle, reins, stirrups → context management, tool use, feedback loops



Harnesses: The Orchestration Layer

A **harness** wraps a frontier model with everything it needs to do useful work:



Tools – file editing, shell commands, web search, MCP integrations

Context – what the model sees, when, and how much

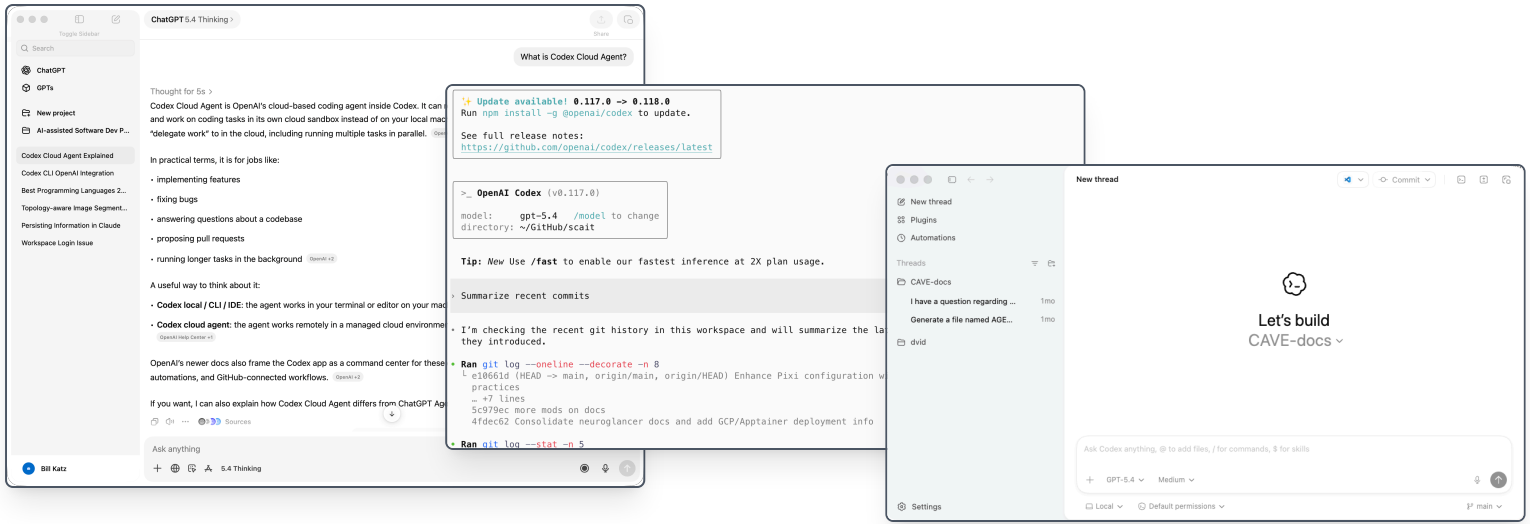
Feedback loops – verification, test results, human review fed back in

Harness UX: Different Surfaces for Different Workflows

UX Surface	What it's for	Claude	OpenAI	Google
IDE	Code in a familiar editor with AI inline	–	Copilot	Gemini Code Assist
CLI	Interactive terminal sessions, agentic coding	Claude Code	Codex CLI	Gemini CLI
Desktop App	Project sidebar, parallel sessions, diff review	Claude App	Codex App	–
Cloud Agent	Async background tasks on your repos	–	Codex Cloud	–
Chat / Web	Conversation-first, paste code in and out	claude.ai	ChatGPT	Gemini

Each vendor is converging on multiple surfaces – but the sweet spots differ.

OpenAI Codex: Different Harnesses → Different UIs



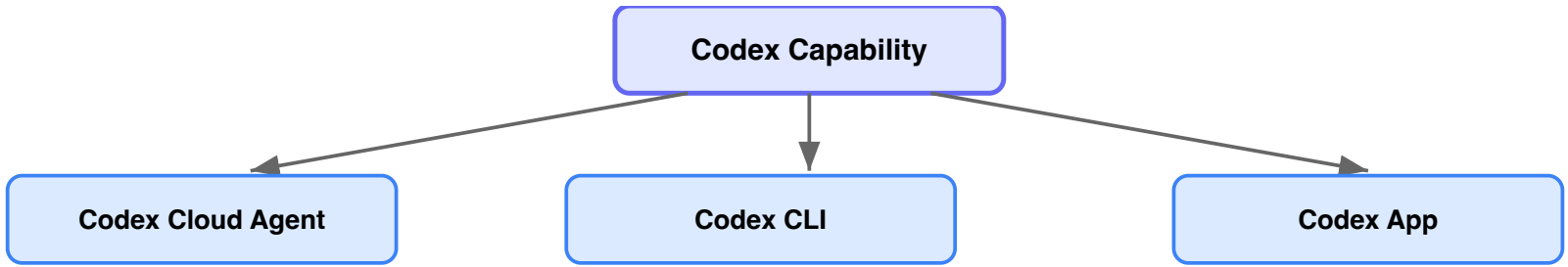
ChatGPT App

Codex CLI

Codex App

OpenAI Codex: Three Control Surfaces

One capability, three interfaces:



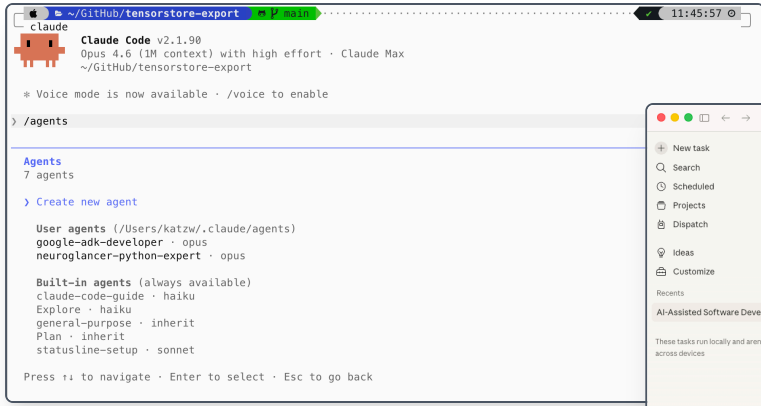
Remote worker
OpenAI-managed environments
Background tasks, parallel execution
Works against your repos

Local terminal interface
Reads, edits, runs code in your dir
Also launches cloud tasks
= local pair programmer + cloud console

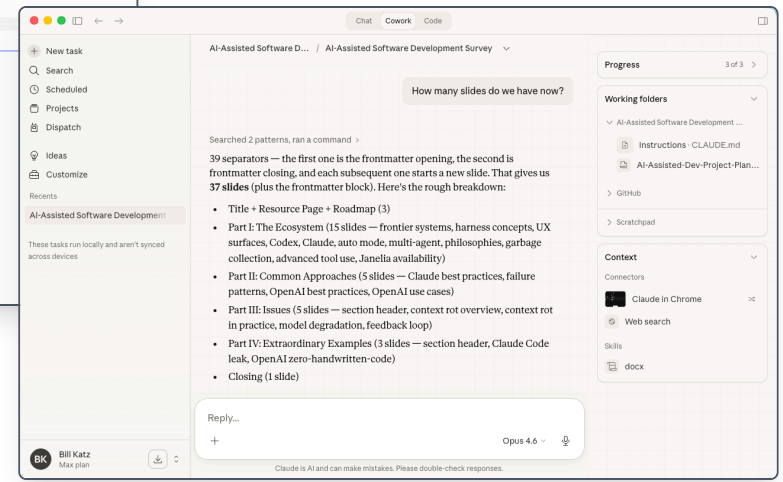
Desktop command center
Supervise parallel Codex threads
Built-in worktrees, Git, automations
Organize work across projects

developers.openai.com/codex

Claude's Two Surfaces → Same Engine, Different UX

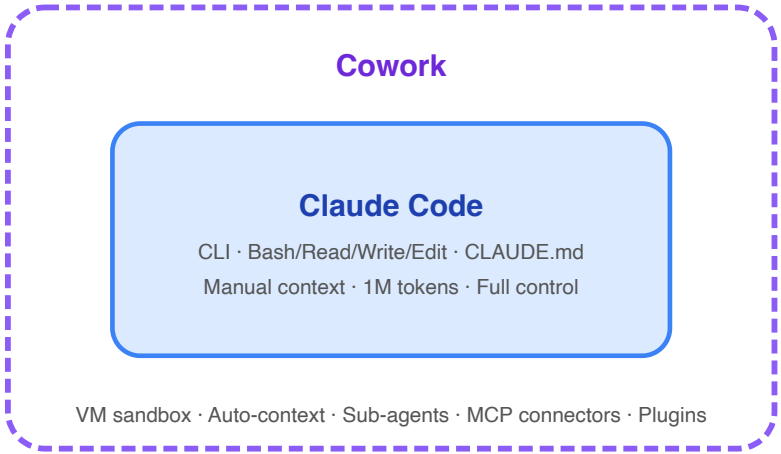


Claude Code (CLI)



Cowork (Desktop)

Claude: Comparison of Two Surfaces



Cowork wraps Claude Code with additional infrastructure for non-developers:

- Full **VM isolation** (host-native virtualization)
- **Automatic** context management – no `/compact` or `/clear`
- **Parallel sub-agents** for complex tasks
- **Three-tier permissions** (none → folder-scoped → explicit approval)
- MCP connectors, plugins, MCP Apps

Built with Claude Code in ~1.5 weeks – humans designed the architecture, agents wrote the code.

Emerging Harness Patterns

Spec-Driven Development

Specs as the **source of truth** – agents code against contracts, not just prompts.

- **GitHub Spec Kit** – Open-source templates for Copilot, Claude Code, Gemini CLI, Cursor, Windsurf
- **Kiro** (Amazon/AWS) – IDE that translates natural language → requirements (EARS notation) → implementation plans → validated code

AI-Powered Design & Prototyping

From **natural language or mockups** directly to working apps.

- **Google Stitch** – Voice-driven "vibe design" canvas, generates HTML/CSS/Tailwind, exports to Figma
- **Antigravity** (Google AI Studio) – Full-stack agent: text prompts → production-ready web apps with auth & data
- **Firebase Studio** – Screenshots or descriptions → working apps from framework templates

The trend: Harnesses are moving upstream – from code generation to requirements capture and UI design.

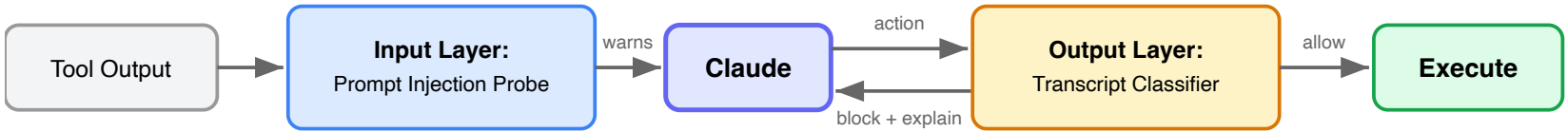
Claude Code Auto Mode: Safety vs. Usability

Users approve **93%** of permission prompts – approval fatigue is real.

anthropic.com/engineering/claude-code-auto-mode

Harness Engineering: Claude Code Auto Mode

A two-layer classifier system instead of "approve everything" or "block everything":



What it catches: scope escalation, credential exploration, unauthorized data sharing, security bypass attempts

What it misses (17% false-negative rate): ambiguous "blast radius" – e.g., does "clean up the PR" authorize a force-push?

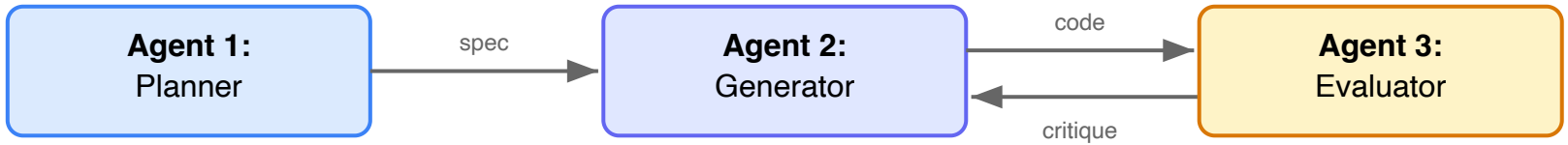
Design choice: strip assistant messages from classifier input so the agent can't talk the classifier into approving dangerous actions.

anthropic.com/engineering/claude-code-auto-mode

Harness Design: Multi-Agent Patterns

Anthropic's Prithvi Rajasekaran on building long-running coding agents:

A single agent evaluating its own work is too optimistic. Solution: **three separate agents**, inspired by GANs:



- One agent alone ($9, 20 min) → broken, unusable code
- Three-agent harness ($200, 6 hrs) → fully working applications
- Models exhibit "context anxiety" – wrapping up early as context fills
- Context *resets* work better than context *compaction* for long tasks

"Find the simplest solution possible, and only increase complexity when needed."

anthropic.com/engineering/harness-design-long-running-apps

Two Philosophies of Harness Design

	Claude Code (Anthropic)	**Codex** (OpenAI)
Core idea	"Bash is all you need"	"The repo is the knowledge store"
Tools	Minimal: Bash, Read, Write, Edit, Glob, Grep	Sandboxed envs, linters, execution plans, observability
Context source	File system – model navigates with basic tools	Repository – if it's not checked in, it doesn't exist
Harness role	Thin wrapper; trust the model	Rigid architecture with enforced boundaries + automated cleanup
Demand on dev	Write good code; model figures it out	Push all knowledge into repo: docs, plans, decisions, specs
Key file	`CLAUDE.md` – project instructions	`AGENTS.md` as table of contents → structured `docs/` tree

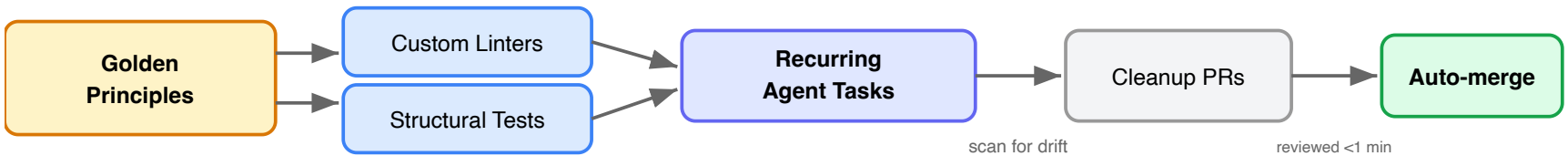
Both work. Different bets on where the intelligence lives.

openai.com/index/harness-engineering

"Garbage Collection" for Technical Debt

OpenAI's team spent 20% of their week manually cleaning up "AI slop." That didn't scale.

Their solution: **encode taste as rules, then automate enforcement.**



- Enforce architecture mechanically: dependency directions, naming conventions, file size limits
- Linter error messages written as remediation instructions for agents
- Technical debt paid continuously in small increments, not painful bursts
- Human taste captured once, enforced on every line of code going forward

openai.com/index/harness-engineering

The Harness Layer Is Evolving Fast

Harnesses are getting smarter about what the model sees and how it acts.

Example: Anthropic's "Advanced Tool Use" (beta, March 2026)

Problem	Solution	Impact
Too many tools loaded upfront	**Tool Search** – find tools on-demand	85% fewer tokens
Too many round-trips to the model	**Programmatic Calling** – orchestrate in code	37% fewer tokens
Schema alone isn't enough	**Tool Use Examples** – show, don't just describe	72% → 90% accuracy

anthropic.com/engineering/advanced-tool-use

What's Available at Janelia

OpenAI (HHMI Enterprise)

- Organization-wide token pool for HQ+Janelia
- ChatGPT, Codex cloud agent, Codex CLI
- It's a big pool. HQ more ChatGPT-oriented.

Anthropic (Claude)

- Enterprise collaboration in progress
- Currently: Claude Max plans or API access
- Soon: Enterprise individualized "consumption" model (not Max)

Google (Gemini)

- Limited Enterprise licenses (~$20/mo Pro) available through Support
- Licensing more fractured. Unsure about Ultra and above.
- Gemini CLI is free with personal Google accounts (1K requests/day)

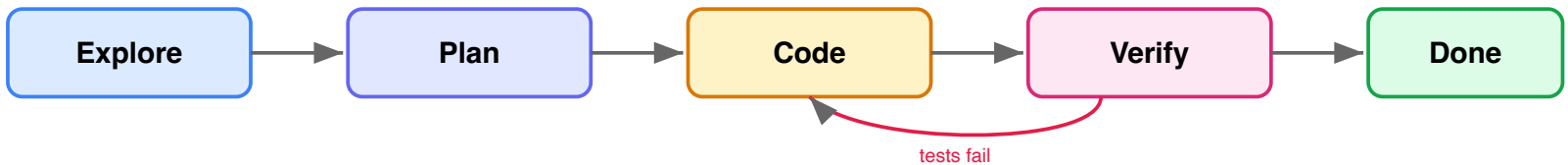
Disclaimer:
As of Apr 2, 2026

Part II: Common Approaches

Patterns for AI-Assisted Development

Claude Best Practices: Explore → Plan → Code → Verify

The single highest-leverage pattern across all harnesses:



Explore – Read code, ask questions, understand the system. Use Plan Mode or Ask Mode.

Plan – Create an implementation plan *before* writing code. Edit the plan yourself if needed.

Code – Let the agent implement against the plan. Reference specific files and patterns.

Verify – Give the agent a way to check its own work: tests, screenshots, linters, build commands. This is the single highest-leverage thing you can do.

Skip the plan for small, obvious tasks. Use it when scope is unclear or changes span multiple files.

code.claude.com/docs/en/best-practices

Claude Hierarchy of Instructions

> Can you describe the hierarchy of instructions you receive? There's Anthropic's instructions, your system instructions (presumably Anthropic's), the various levels of my `CLAUDE.md` in different parts of the file system (please elaborate how many there can be), and then what I'm typing into the prompt here with you, and possibly others which I want you to list.

● Here's the full hierarchy, roughly from highest to lowest priority:

1. **Anthropic's safety instructions** — Hardcoded in the system prompt. Can't be overridden by anything. Things like refusing to help with malicious attacks, not generating harmful content.

2. **Claude Code system instructions** — The large system prompt that defines how I use tools, commit practices, tone, output efficiency, etc. Also from Anthropic, delivered at conversation start.

3. **Your live conversation input** — What you type in the prompt. This is the most current context and generally takes precedence over persistent configuration for the task at hand.

4. **CLAUDE.md files** — There are multiple levels, loaded in order:
- `~/.claude/CLAUDE.md` — User-level, applies across all your projects
- `{project_root}/CLAUDE.md` — Project-level, checked into the repo (shared with collaborators)
- `{project_root}/.claude/CLAUDE.md` — Project-level local, gitignored (your personal overrides for this project)
- Parent directories between `~` and the project root can also have CLAUDE.md files

   The system prompt says these "OVERRIDE any default behavior and you MUST follow them exactly as written."

5. **Hook feedback** — Shell commands you configure in `settings.json` that run in response to events (tool calls, prompt submission). Treated as coming from you.

6. **Memory files** — My persistent memory at `~/.claude/projects/{project}/memory/`. Supplementary context, explicitly lower priority than CLAUDE.md and current observations.

7. **System reminders** — Injected tags during the conversation (date changes, task reminders, skill availability, plan mode). These are from the Claude Code harness, not from you directly.

8. **Skills** — Slash commands that expand into specialized prompts when invoked.

The key tension is between (3) and (4) — your live input can override CLAUDE.md for the current task, but CLAUDE.md represents your durable preferences. In practice I try to follow both, and ask when they conflict.

Common Failure Patterns (and Fixes)

Recognizing these early saves hours:

The kitchen sink session – One task, then something unrelated, then back to the first. Context fills with noise. → `/clear` between unrelated tasks.

Correcting over and over – Claude does it wrong, you correct, still wrong, correct again. Context polluted with failed approaches. → After two failed corrections, `/clear` and write a better initial prompt.

The over-specified CLAUDE.md – Too long, Claude ignores half of it. Important rules get lost. → Ruthlessly prune. If Claude already does it correctly without the rule, delete it.

The trust-then-verify gap – Plausible-looking code that doesn't handle edge cases. → Always provide verification. If you can't verify it, don't ship it.

The infinite exploration – "Investigate this" without scoping. Claude reads hundreds of files. → Scope narrowly, or delegate to a subagent so exploration doesn't consume your main context.

code.claude.com/docs/en/best-practices

OpenAI Best Practices for AI-Assisted Coding

From OpenAI's internal engineering teams — but these generalize across harnesses:

Start with a plan, then code — Use "Ask mode" first to generate an implementation plan. Then switch to coding mode with that plan as input. Keeps the agent grounded.

Structure prompts like GitHub Issues — Include file paths, component names, diffs, and doc snippets. "Implement this the same way it's done in module X" improves results.

Use the task queue as a backlog — Fire off tangential ideas, partial work, incidental fixes. No pressure for a full PR in one go. Review when you're back in focus.

Iterate on the environment, not the prompt — Build errors? Fix the dev environment config, not the prompt. Startup scripts, env vars, and internet access reduce error rates significantly.

"Best of N" for hard problems — Generate multiple responses for a single task, review several iterations, combine the best parts.

Keep tasks well-scoped — Sweet spot: tasks that would take you about an hour or a few hundred lines of code.

OpenAI: "How OpenAI uses Codex" (PDF)

How OpenAI Engineers Actually Use Codex

Seven use cases from OpenAI's internal teams (Security, Frontend, API, Infrastructure):

Use Case	What they do
Code understanding	Paste a stack trace, ask where the auth flow lives – faster than grep
Refactoring & migrations	Swap legacy patterns across dozens of files, open the PR in minutes
Performance optimization	Flag hot paths, draft batched queries – "5 min prompt saves 30 min work"
Test coverage	Point at low-coverage modules overnight, wake up to unit-test PRs
Development velocity	Scaffold boilerplate, triage bugs, ship low-priority fixes from backlog
Staying in flow	Fire off drive-by fixes as background tasks, review PRs when free
Exploration & ideation	Explore alternative architectures, find similar bugs across codebase

These patterns apply equally to Claude Code, Gemini CLI, and other harnesses.

OpenAI: "How OpenAI uses Codex" (PDF)

Case Study: gstack – A Shareable Skills Library

Garry Tan (YC CEO) open-sourced **23 slash-command skills** built on the **SKILL.md standard** – works with any agent that supports it (Codex CLI, Claude Code, Gemini CLI, Cursor, Factory Droid).

Think → Plan → Build → Review → Test → Ship → Reflect

Role	Skill
YC Office Hours	`/office-hours` – reframes problem before coding
CEO / Founder	`/plan-ceo-review` – challenges scope
Eng Manager	`/plan-eng-review` – locks architecture
Staff Engineer	`/review` – finds bugs that pass CI
QA Lead	`/qa` – real browser testing
Release Engineer	`/ship` – tests, coverage audit, PR
Security Officer	`/cso` – OWASP + STRIDE audits

Why it matters:

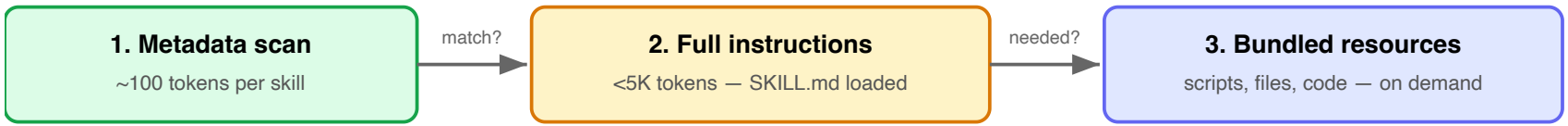
- Skills are **Markdown files** in `.agents/skills/` – portable, forkable, composable
- Agent-agnostic via the **SKILL.md standard** (`--host codex` , `--host auto`)
- Encodes **process as code** – the sprint structure is the product
- Supports 10–15 parallel agent sessions
- 62K+ stars, 8K+ forks, MIT licensed

The pattern: Capture your team's engineering taste once, let agents enforce it on every line of code.

github.com/garrytan/gstack

Claude Skills: Progressive Disclosure Architecture

Skills are specialized folders (instructions, scripts, resources) that teach Claude to perform tasks repeatably – available across Claude.ai, Claude Code, and the API.



Multiple skills stay available without overwhelming the context window

Official skills: docx, pdf, pptx, xlsx, algorithmic-art, canvas-design, mcp-builder, webapp-testing, skill-creator

Community: awesome-claude-skills – curated directory of skills, tools, and resources

Providing Large Codebase Context to AI

Two dominant patterns for helping AI understand codebases too large for any context window:

RAG (Retrieval-Augmented Generation)

Embed code chunks as vectors, retrieve by semantic similarity at query time.
Used by: GitHub Copilot (code search + RAG), Augment Code (knowledge graph + RAG)

✓ Automated – no human curation needed

✓ Scales to large, changing codebases

✗ Chunking destroys structural relationships

✗ Finds *similar-looking* code, not *architecturally-related* code

Hierarchical Maps (llms.txt / CLAUDE.md)

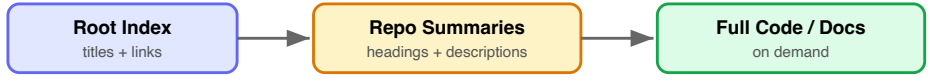
Human-curated indexes at multiple levels of detail. Agent navigates from summaries to specifics.
Used by: Claude Code (docs map), llmstxt.org standard, CLAUDE.md files

✓ Preserves structure and relationships

✓ Agent navigates purposefully, not blindly

✗ Requires curation and maintenance

✗ Coverage depends on author effort



Maps: progressive detail — agent reads only what it needs

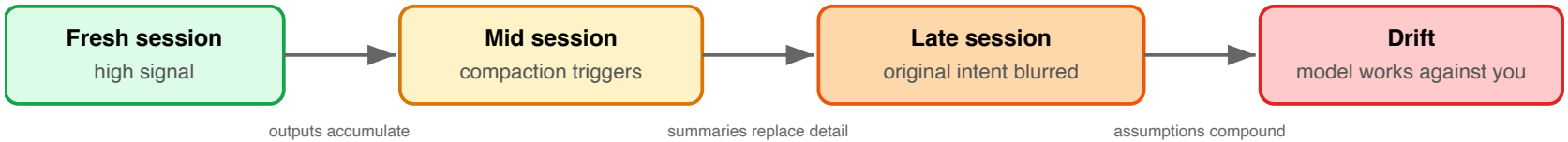
llmstxt.org · Simon Willison on Claude Code's docs map · Claude Code docs map

Part III: Issues

Context Rot, Model Degradation, and the Feedback Loop

Context Rot: Death by a Thousand Tokens

As a session grows, the model's effective intelligence degrades – not because it gets dumber, but because signal gets buried in noise.



What rots:

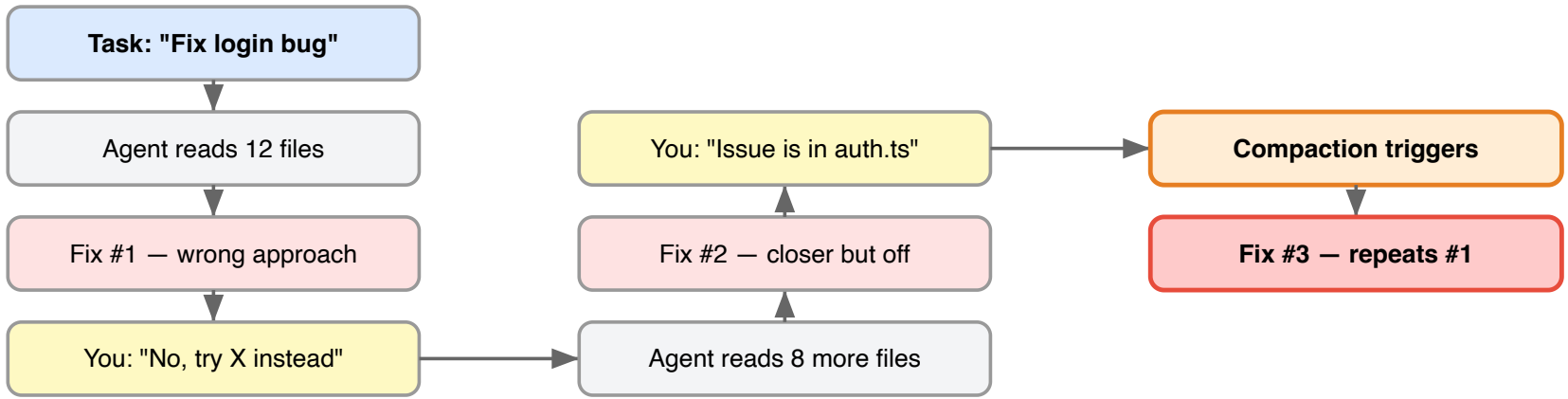
- Original intent and constraints get summarized away
- Failed approaches stay in context, biasing toward same mistakes
- Tool outputs crowd out your instructions
- After compaction: summary of a summary

What helps:

- Start fresh (`/clear`) for genuinely new tasks
- Keep sessions focused on one goal
- Front-load constraints in CLAUDE.md (survives compaction)
- Use subagents so main context stays clean

Context Rot in Practice

A real pattern that wastes hours:



Context is now ~80% failed attempts. The model is optimizing against polluted history.

The fix: After two failed corrections, `/clear` and write a single, precise prompt incorporating what you've learned. Five minutes of prompt writing beats thirty minutes of correction loops.

Model Degradation: Why Does It Feel Dumber Sometimes?

The model is a black box. When quality seems to drop, the cause is rarely what you think.

It's probably not the model:

- **Context pollution** – noise accumulated over a long session drowns out your intent
- **Prompt ambiguity** – your request is clear to you but underspecified for the model
- **Compaction artifacts** – after auto-compaction, the model lost critical details
- **Task complexity creep** – what started small became multi-file, multi-concern

It might be the model:

- **Model version changes** – providers update models without notice; behavior shifts
- **Capacity-dependent routing** – some providers route to smaller models under load
- **Regression in specific domains** – a new model version may improve overall but regress on your particular use case

When in doubt: start a fresh session and test with a clean prompt.

The Feedback Loop: Anthropic Is Listening

Anthropic tracks user satisfaction signals from conversations – including frustration.

When you curse at Claude, that interaction is flagged as a **negative satisfaction signal**. Claude doesn't change its behavior in response to cursing, but the data feeds back into model evaluation and training priorities.

What Anthropic captures:

- Thumbs up/down on responses
- Conversation-level sentiment (including profanity as frustration signal)
- Retry and regeneration patterns
- Session abandonment timing

Why this matters for developers:

- Your frustration literally shapes the next model version
- Specific, constructive feedback (thumbs down + explanation) is far more actionable than cursing
- The 👎 button is your most powerful tool for improving AI coding assistants

Part IV: Extraordinary Examples

What People Are Actually Building

The Claude Code Leak → Agent-Driven Rewrites

A missing `.npmignore` shipped 512K lines of unobfuscated TypeScript in the npm package. What happened next:

The leak (March 31, 2026):

- Anthropic: "a release packaging issue caused by human error, not a security breach"
- Sigrid Jin (@realsigridjin), exhausted after running two Korean hackathons in SF, cloned the repo, went to sleep
- Woke up to massive attention — and his girlfriend (a copyright lawyer) begging him to take it down

The pivot:

- Team decision: "What if we have agents **rewrite the entire thing in Python**? Surely that's more legal…"
- Clean-room Python rewrite using **oh-my-codex** (OmX), an AI workflow tool built on OpenAI Codex
- Board a plane to South Korea
- Mid-flight, one team member decides Python is too slow — starts rewriting **all of it into Rust**

The result: "claw-code"

- **50K GitHub stars in two hours** — fastest-growing repo in GitHub history
- Anthropic filed **8,100+ DMCA takedowns**
- The knock-off now has more stars than the original Claude Code repo

OpenAI's Zero-Handwritten-Code Experiment

An OpenAI team built and shipped a product with **0 lines of manually-written code**.

Metric	Value
Hand-written code	0 lines
Agent-generated code	~1 million lines
PRs merged	~1,500
Engineers	3 (later 7)
Throughput	3.5 PRs / engineer / day
Estimated speedup	~10x vs hand-coding
Duration	5 months (from empty repo, ~Sep 2025 – Feb 2026)

The product has internal daily users and external alpha testers. It ships, deploys, breaks, and gets fixed — all by agents.

Engineers spent 20% of time cleaning up "AI slop" → solved by automated "garbage collection" (golden principles + recurring agent cleanup tasks).

openai.com/index/harness-engineering

Pretext: AI-Friendly Iteration Against Browser Ground Truth

Cheng Lou's **Pretext** – a JS library that measures and lays out multiline text *without* DOM manipulation, using pure computation.

The problem: Measuring text in the browser requires triggering expensive DOM reflows – a bottleneck for any UI that needs text height, line counts, or variable-width layouts.

The approach:

- `prepare()` measures text segments once using the canvas font engine as ground truth (~19ms for 500 texts)
- `layout()` computes height, lines, and wrapping with pure arithmetic (~0.09ms for 500 texts)
- No DOM in the hot path – works with Canvas, SVG, or WebGL

Why it's here – the development process:

Parallel agents trained against real browser rendering for weeks, iterating over and over until output matched Safari, Chrome, and Firefox exactly.

Browser engines provide pixel-perfect ground truth → agents iterate and verify autonomously in parallel → humans focus on architecture, agents grind on correctness.

Result: Supports all languages, emoji, bidi text, `pre-wrap`, shrink-wrap sizing, and text flowing around floats.

github.com/chenglou/pretext

Resources & Discussion

GitHub: `JaneliaSciComp/ai-assisted-dev`

Explore the curated resource list and contribute links.